BPFize Your Kernel Subsystem
The fanotify Experience

Song Liu <song@kernel.org>

Kernel Recipes 2025

Meta

Who am I?

- Song Liu
- Software engineer @ Meta
- Linux kernel contributor, reviewer, and maintainer
- Work with BPF for 7+ years
- Support BPF LSM users @ Meta

What is BPF/eBPF?

- BPF originally stood for Berkeley Packet Filter
- eBPF means extended BPF
- BPF and eBPF are generally used interchangeably



What is BPF/eBPF?



- BPF originally stood for Berkeley Packet Filter
- eBPF means extended BPF
- BPF and eBPF are generally used interchangeably

- How would you describe BPF for someone who has never heard of it?



“There are a thousand Hamlets in a thousand people’s eyes.”

“一千个人眼中有一千个哈姆雷特。”

Well-known proverb in China

“There are a thousand BPFs on a thousand kernel hackers’ keyboards.”

“What is BPF in my eyes? How do I use BPF on my keyboard?”

One of those 1000 kernel hackers.



01 **What is BPF?**

02 BPFize your kernel subsystem

03 BPF LSM

04 fanotify-bpf

“eBPF is a revolutionary technology with origins in the Linux kernel that can run sandboxed programs in a privileged context such as the operating system kernel. It is used to safely and efficiently extend the capabilities of the kernel without requiring to change kernel source code or load kernel modules.”

https://ebpf.io/what-is-ebpf/



Something

is a tasty food from France

that goes well with pesto source

Something

is a tasty food from France

that goes well with pesto source

and is often served in its own shell

Something

is a tasty food from France

that goes well with pesto source

and is often served in its own shell



“eBPF is a revolutionary technology with origins in the Linux kernel that can run sandboxed programs in a privileged context such as the operating system kernel. It is used to safely and efficiently extend the capabilities of the kernel without requiring to change kernel source code or load kernel modules.”

https://ebpf.io/what-is-ebpf/



“eBPF is a revolutionary technology with origins in the Linux kernel that can run **sandboxed** programs in a **privileged context** such as the operating system kernel. It is used to **safely** and efficiently **extend the capabilities** of the kernel without requiring to change kernel source code or load kernel modules.”

https://ebpf.io/what-is-ebpf/



BPF brings programmability

- Traditionally, we can only tune kernel behavior with knobs: sysfs, syscalls, sysctl, etc.
- BPF Instruction Set Architecture (ISA)

BPF is powerful and performant

- Running in the privileged contexts, BPF have visibility of events happened in the whole stack
- XDP (eXpress Data Path) runs in the device driver to achieve extreme performance

BPF is safe

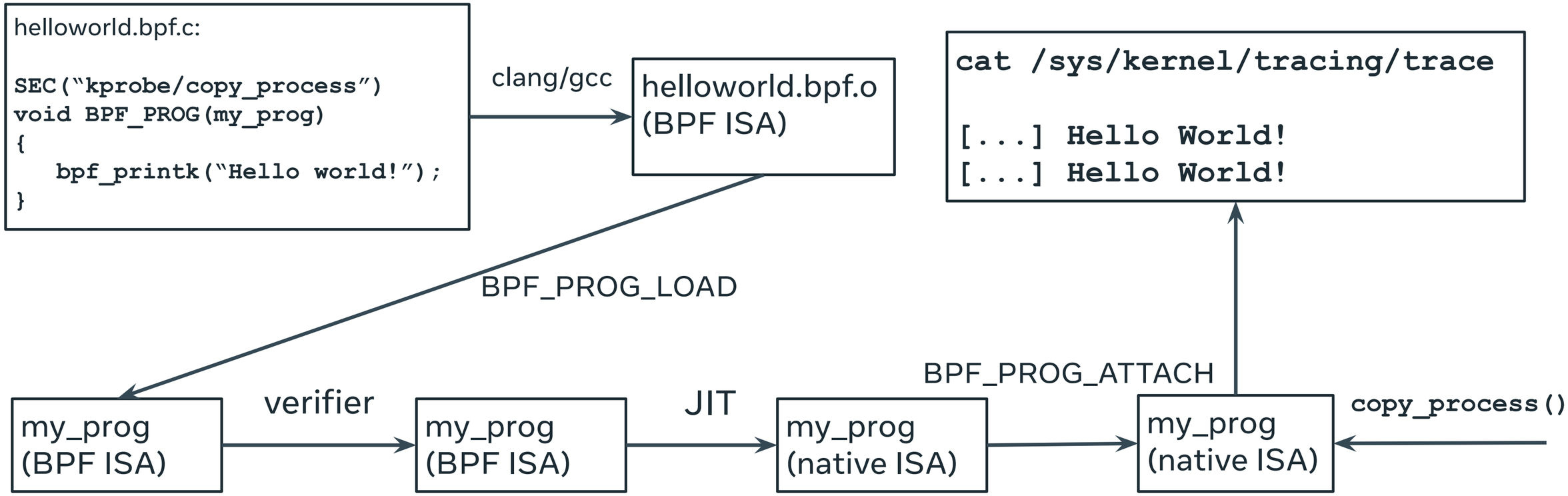
- BPF will not crash the system
- BPF are guaranteed to terminate
- BPF will not read uninitialized memory
- BPF will not leak memory
- BPF will not leak reference to kernel objects
- and the list is growing

BPF is safe, but how?

- BPF verifier checks the safety for each BPF program
- Verifier works with BPF instructions
  - This is one of reason why we need BPF ISA
- What about performance? JIT!



BPF program from source to execution



Sandboxed vs. privileged context

- This “sandbox” runs in the privileged context
- To fully leverage the privileges, BPF programs need reusable building blocks
  - BPF maps, BPF helpers/kfuncs, etc.
- The uses of these building blocks are verified to be safe

Reusable building blocks

- BPF maps => memory management, input/output
- BPF helpers and kfuncs => interact with the system
- BPF iterator, BPF trampoline => trigger BPF programs for certain use cases
- BTF (BPF Type Format) => understand the kernel/BPF data structures

BPF challenges

- Writing verifiable BPF program is not always straightforward
- Creating reusable building blocks is harder than writing code for a single use case



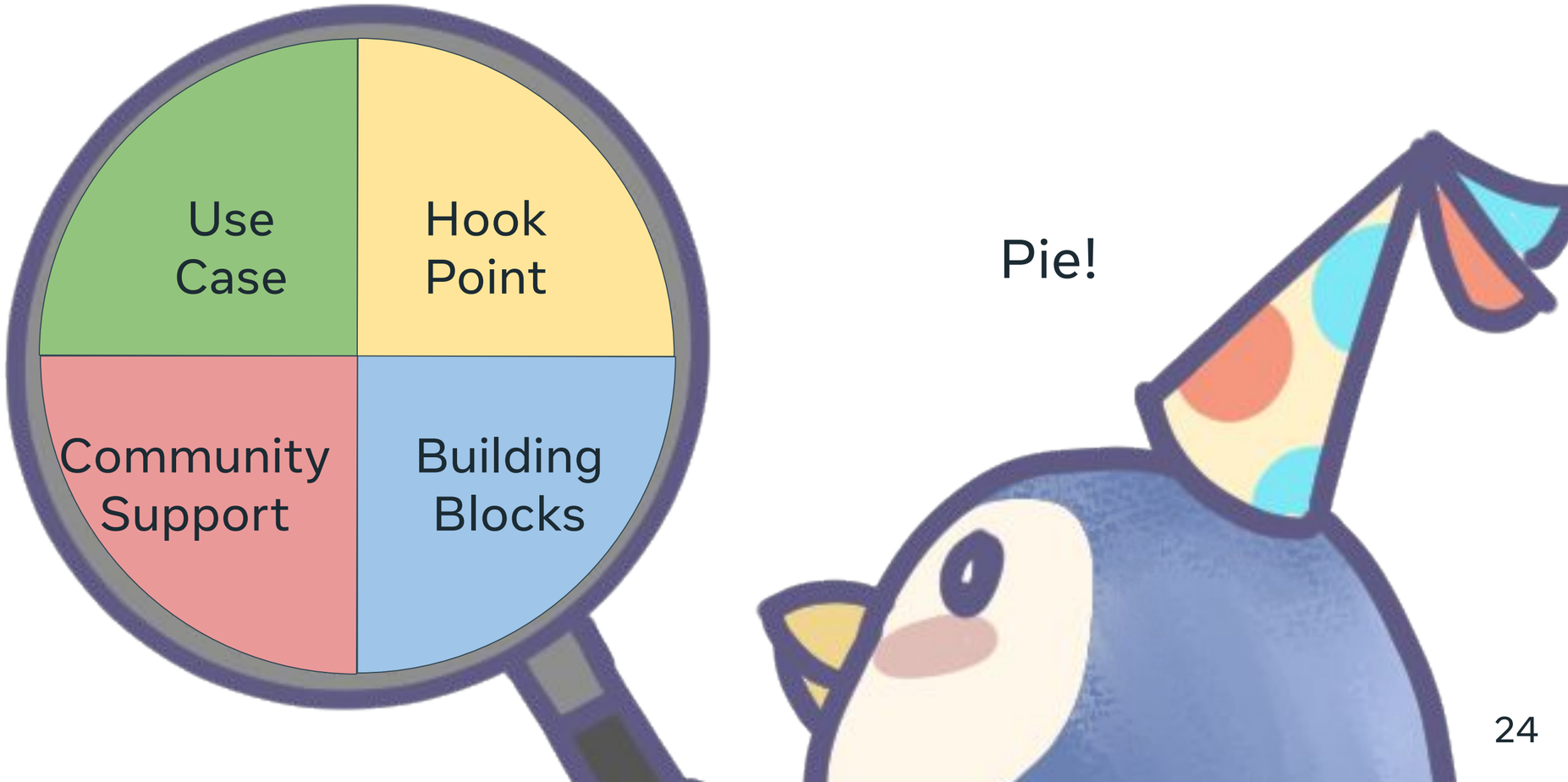
01 What is BPF?

02 **BPFize your kernel subsystem**

03 BPF LSM

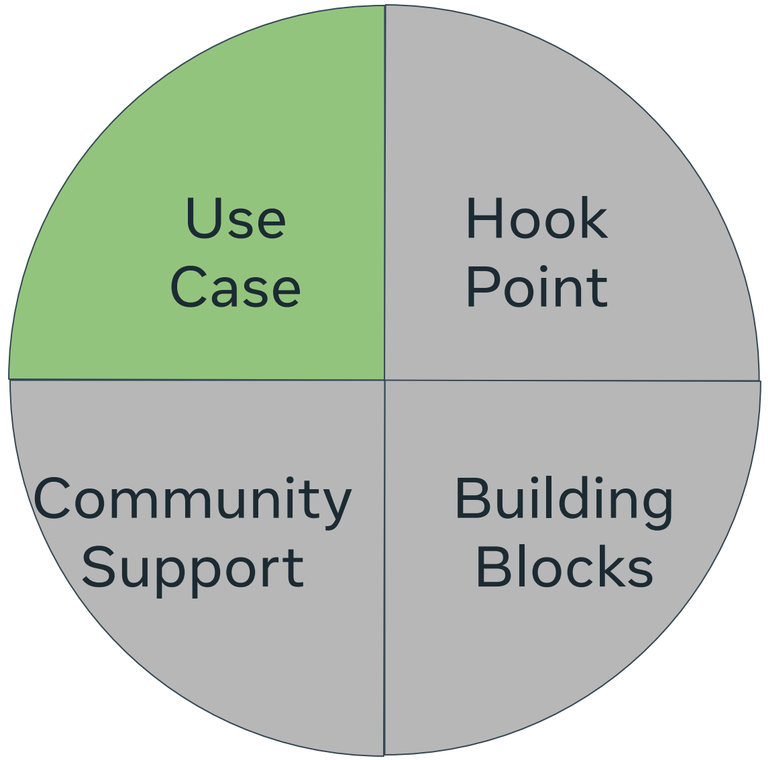
04 fanotify-bpf

BPFize your kernel subsystem



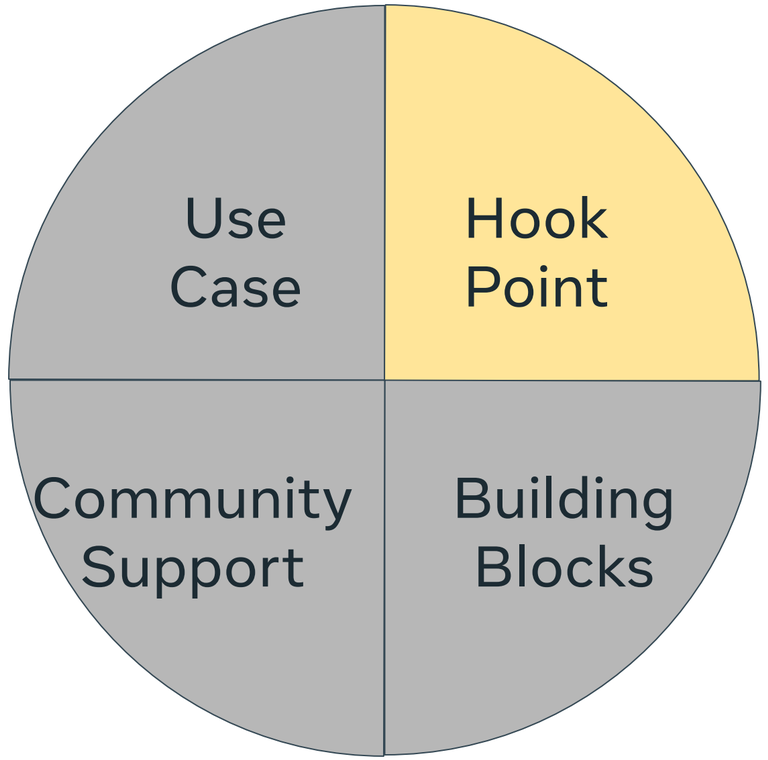
Know your use case

- Why do we need BPF?
  - Programmability? Speed?
- What are the alternatives?
- How do BPF fit in the whole solution?



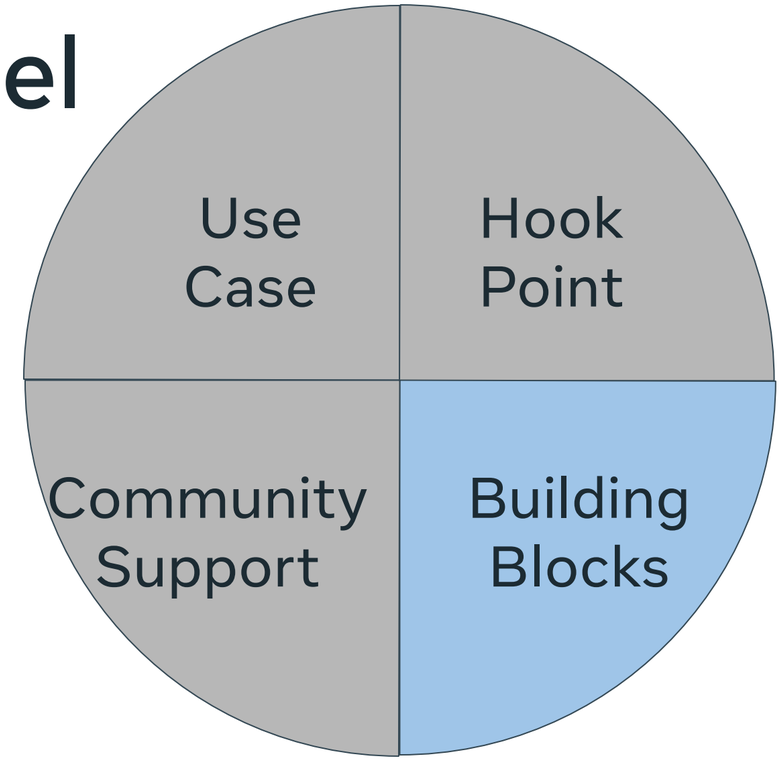
Design your hook point

- ~~Add a pointer to `struct bpf_prog`, and some code to call the program~~
- For most new use cases, struct_ops is the recommended solution
- Try to reuse existing mechanism



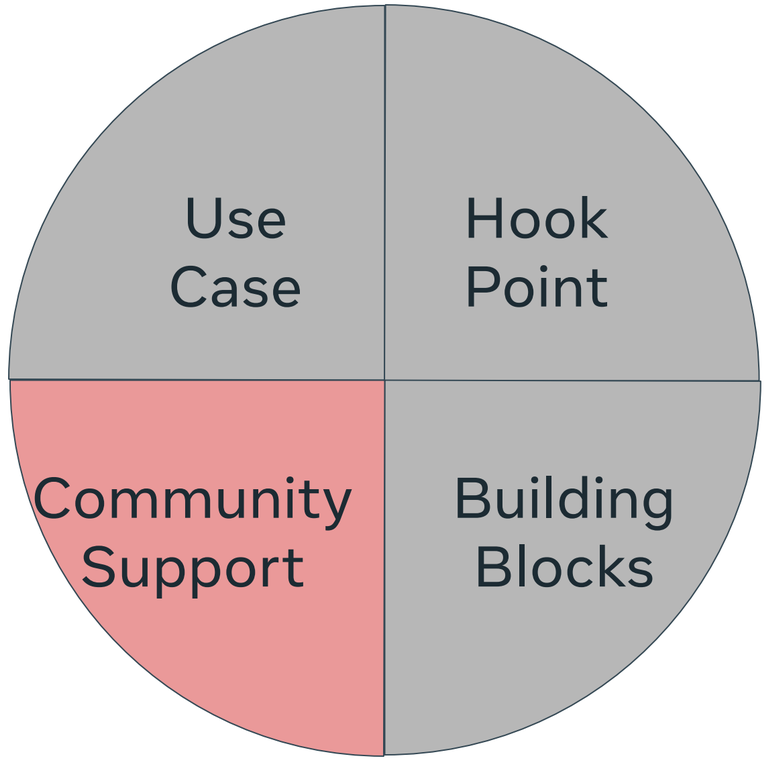
Add building blocks to the kernel

- ~~New BPF helpers~~ New BPF kfuncs
- New map type
- Try to reuse existing building blocks
  - e.g. Use hashmap/array instead of new map type



Work with the community

- Help the community understand the use case
- Be prepared to redesign your solution
- Write self tests for kernel changes
- Support your code in the upstream kernel





01 What is BPF?

02 BPFize your kernel subsystem

03 **BPF LSM**

04 fanotify-bpf

Access control: DAC vs. MAC

- Discretionary Access Control (DAC): The owner have full control of the resources

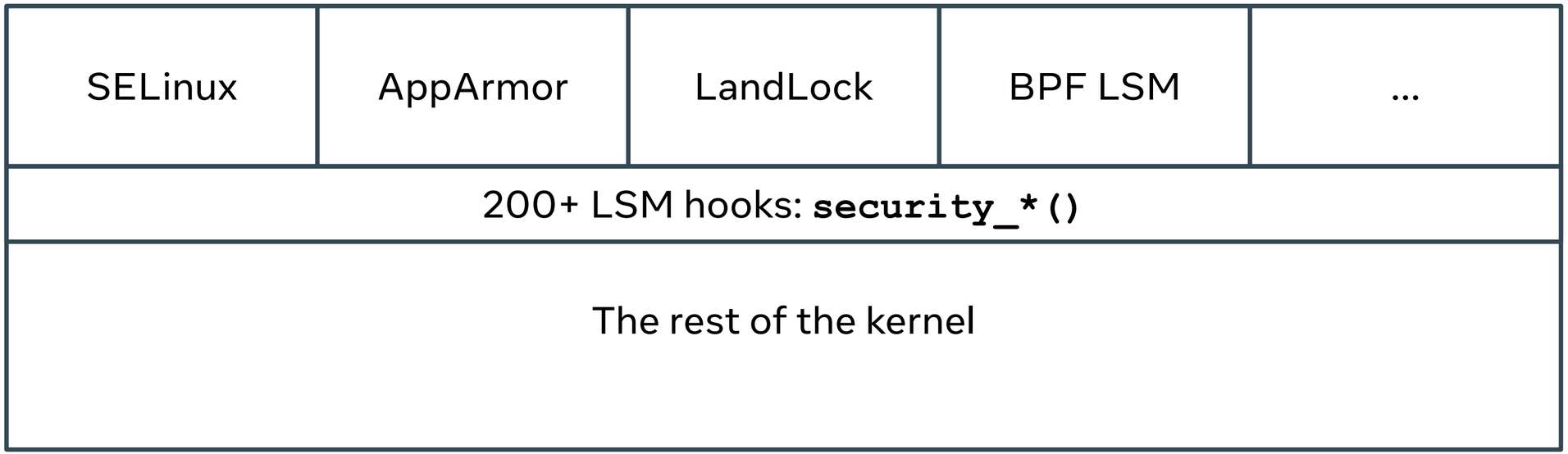
```
ls -l Makefile
-rw-r--r-- 1 song myGroup  7764 Sep 12 16:56 Makefile
```

- Mandatory Access Control (MAC): Detailed rules/policies

```
ALLOW firefox_process firefox_log:FILE WRITE;
```

LSM: Linux Security Module

- Linux's MAC solution



VOID hooks

Notify the LSMs that something happened in the system

```
static inline void file_free(struct file *f)
{
    security_file_free(f);
    /* ... */
}
```

INT hooks

Also allow the LSMs to deny an operation

```
static int do_dentry_open(struct file *f, ...)
{
    /* ... */
    error = security_file_open(f);
    if (error)
        goto cleanup_all;  /* deny open() */
    /* ... */
}
```

LSM hook functions

```
static int apparmor_file_open(struct file *file)
{
    /* Logic that decides whether this open() is allowed */
}

static struct security_hook_list apparmor_hooks[] = {
    /* ... */
    LSM_HOOK_INIT(file_open, apparmor_file_open),
    /* ... */
};
```

Inside a LSM hook

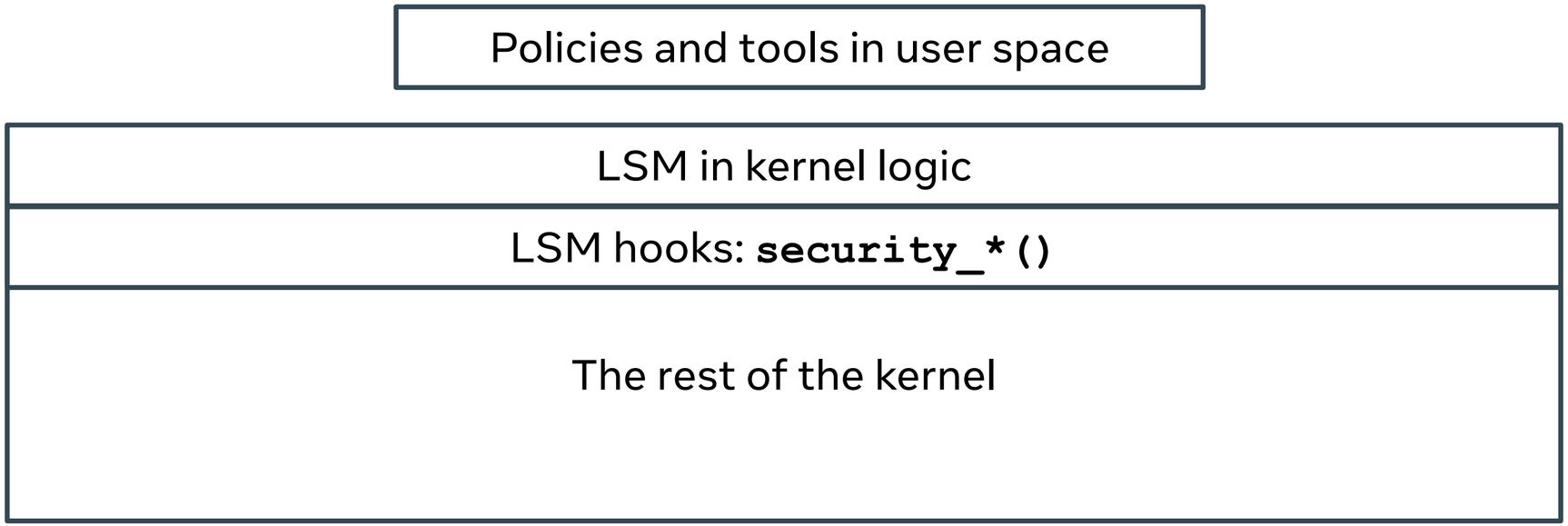
```
int security_file_open(struct file *file)
{
    return call_int_hook(file_open, file);
}
```



```
for_each_lsm(some_lsm) {
    ret = some_lsm_file_open(file);
    if (ret != 0)
        return ret;
}
return 0;
```

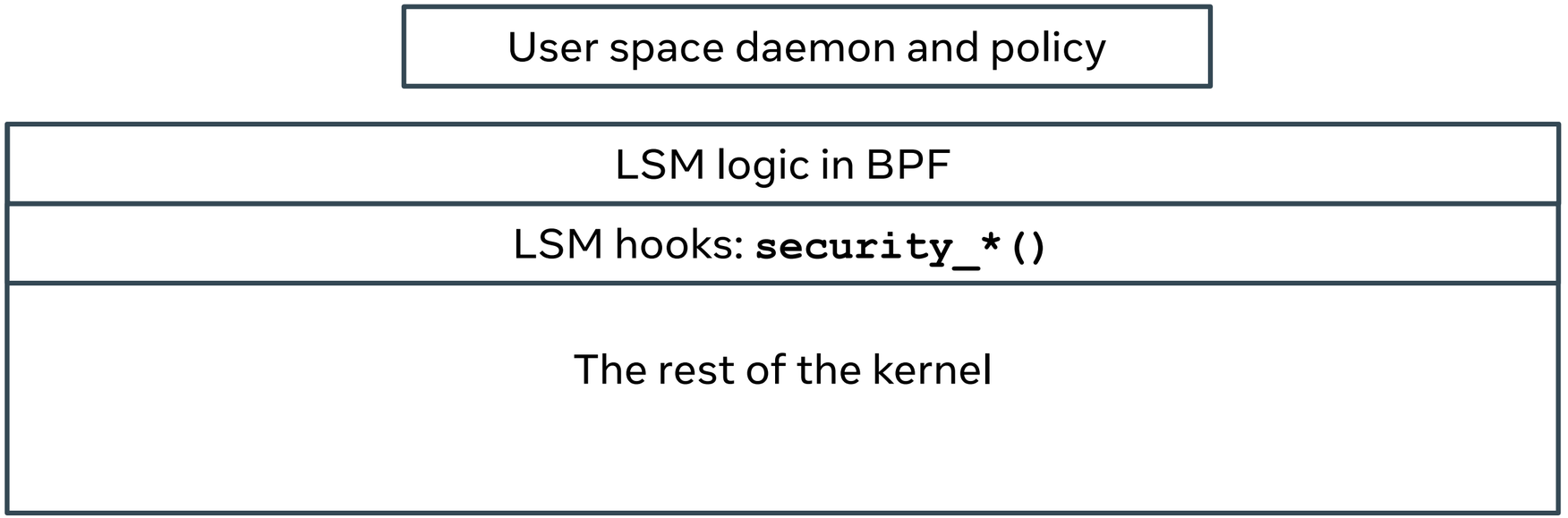
In tree LSM

- In kernel logic to support a policy language
- User space policy sets crafted for given use cases

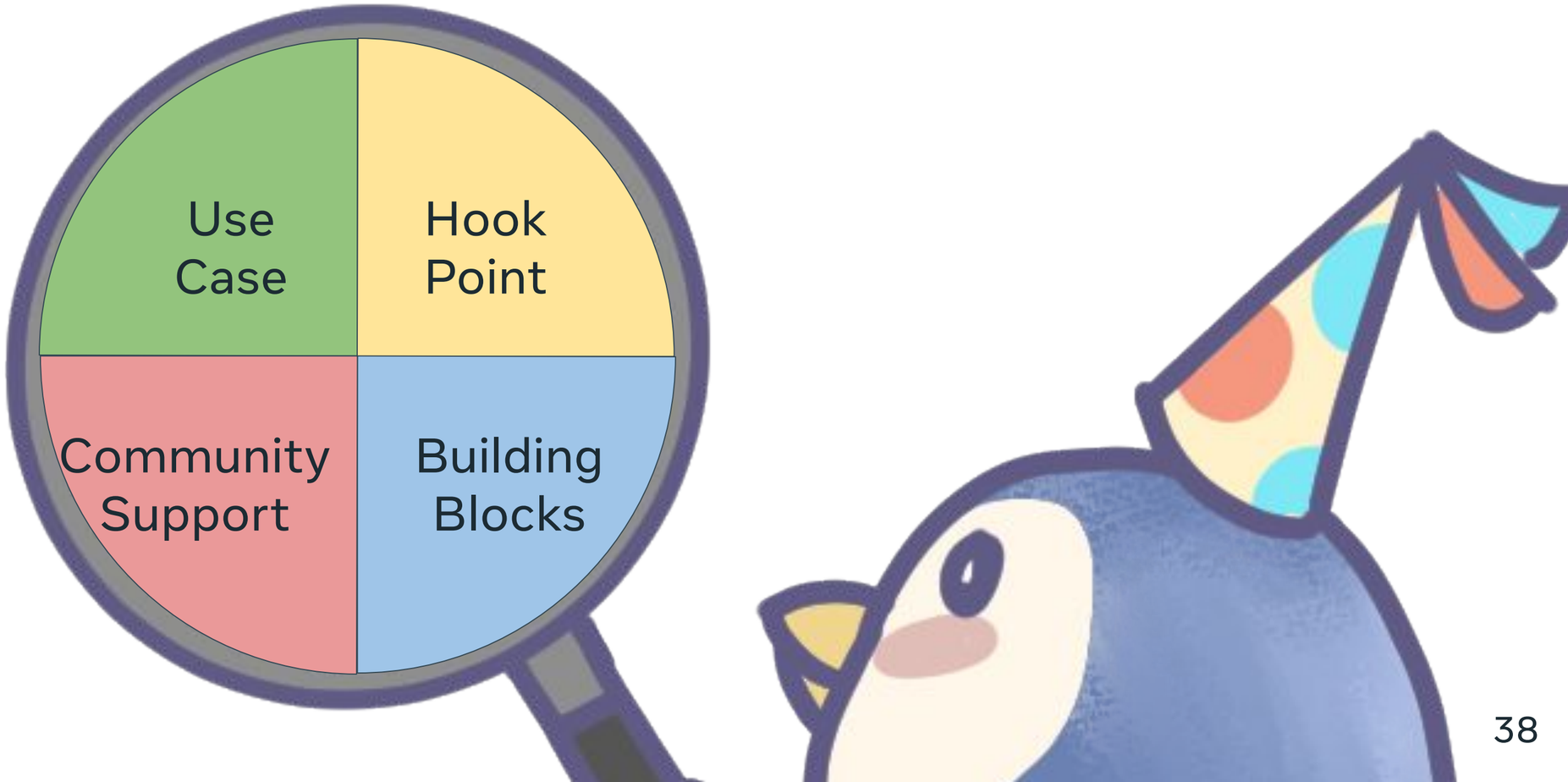


BPF LSM

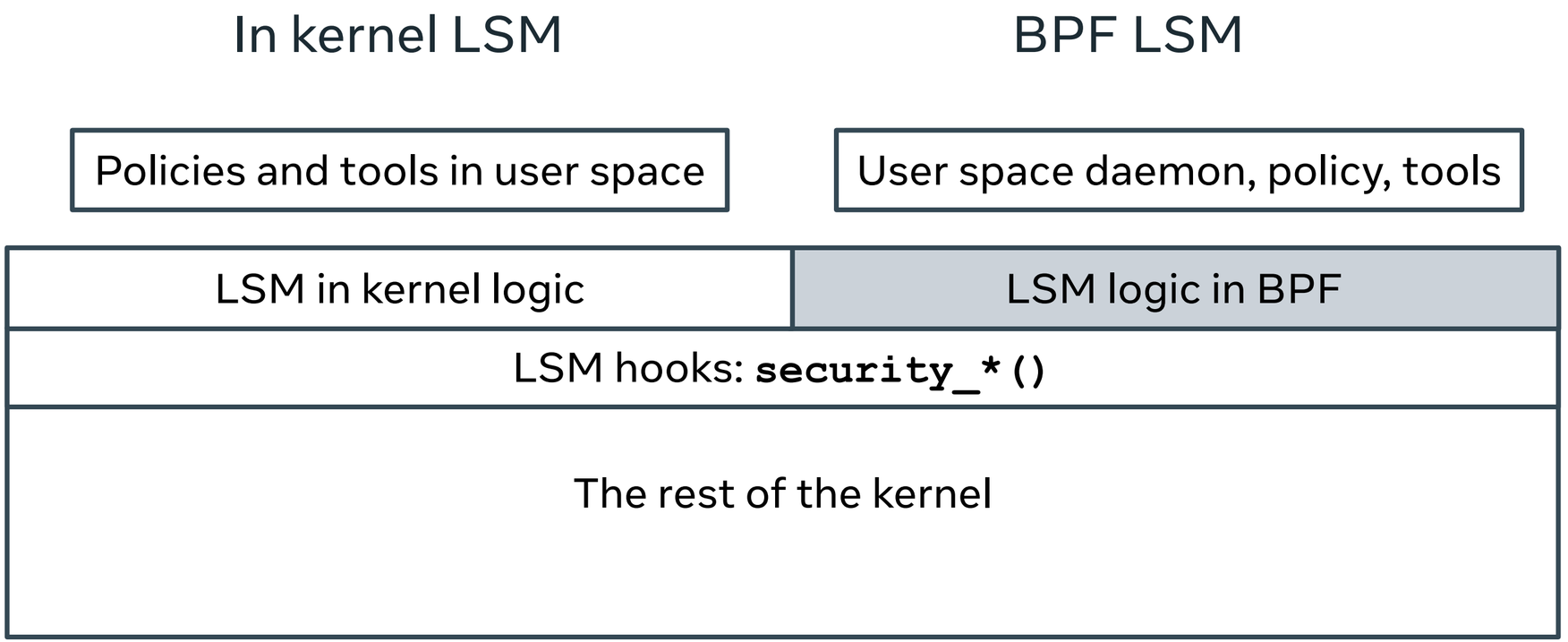
- Kernel space logic written in BPF programs
- User space daemon controls BPF programs and policies



BPF LSM

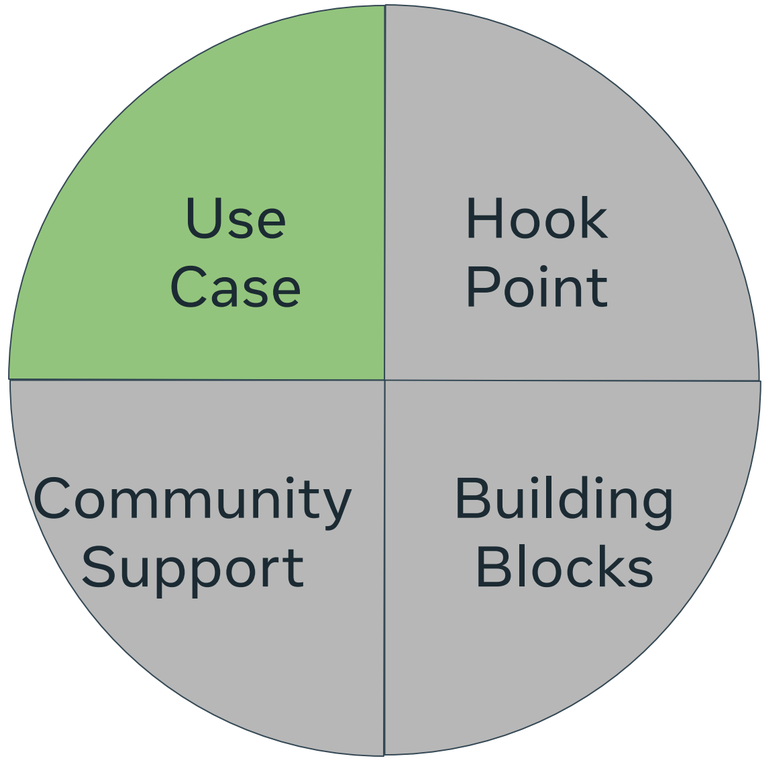


In kernel LSM vs. BPF LSM



BPF LSM use case

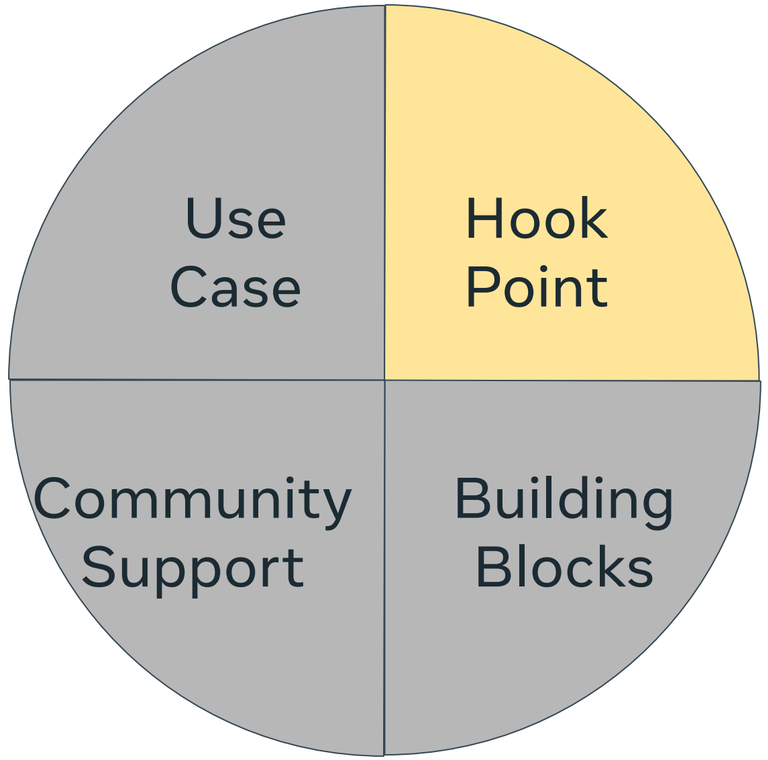
- Flexibility
- Programmability
- Alternative: in tree LSMs



BPF LSM hooks

```
static int bpf_lsm_file_open(struct file *file)
{
    return 0;
}

static struct security_hook_list bpf_lsm_hooks[] = {
    /* ... */
    LSM_HOOK_INIT(file_open, bpf_lsm_file_open),
    /* ... */
};
```

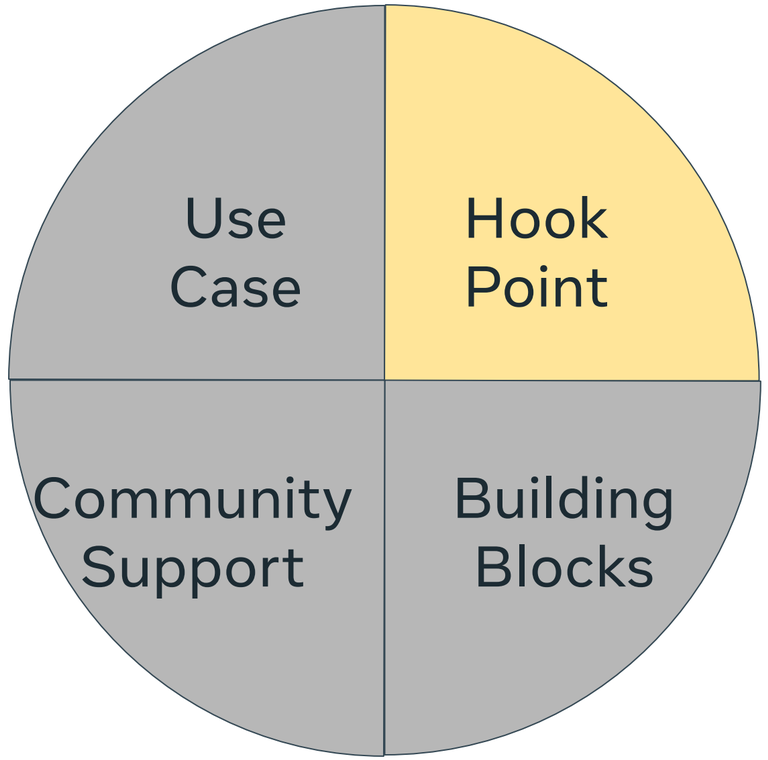


BPF LSM hooks

```
static int bpf_lsm_file_open(struct      le)
{
    return 0;
}

static struct security_hook_list bpf_lsm_hooks[] = {
    /* ... */
    LSM_HOOK_INIT(file_open, bpf_lsm_file_open),
    /* ... */
};
```





BPF trampoline

- A small function generated at run time to call BPF program(s)
  - Use direct calls for fast execution
  - Can reliably access function arguments at function return time
- BPF_TRAMP_MODIFY_RETURN: Allows a BPF program to modify the return value of the target kernel function
  - Used in error injection and BPF LSM

BPF trampoline with BPF_TRAMP_MODIFY_RETURN

```
int bpf_lsm_file_open(struct file *file)
{
    nop(); /* __fentry__ */

    return 0;
}
```

```
bpf_tramp_xxxx(struct file *file)
{
    ret = call(bpf_lsm_file_open+sizeof(nop));
    modified_ret = bpf_prog_xxxx(file, ret);
    return modified_ret;
}
```

BPF trampoline with BPF_TRAMP_MODIFY_RETURN

```
int bpf_lsm_file_open(struct file *file)
{
    nop(); call(bpf_tramp_xxxx, file);

    return 0;
}
```

```
bpf_tramp_xxxx(struct file *file)
{
    ret = call(bpf_lsm_file_open+sizeof(nop));
    modified_ret = bpf_prog_xxxx(file, ret);
    return modified_ret;
}
```

BPF trampoline with BPF_TRAMP_MODIFY_RETURN

```
int bpf_lsm_file_open(struct file *file)
{
    nop(); call(bpf_tramp_xxxx, file);

    return 0;
}
```

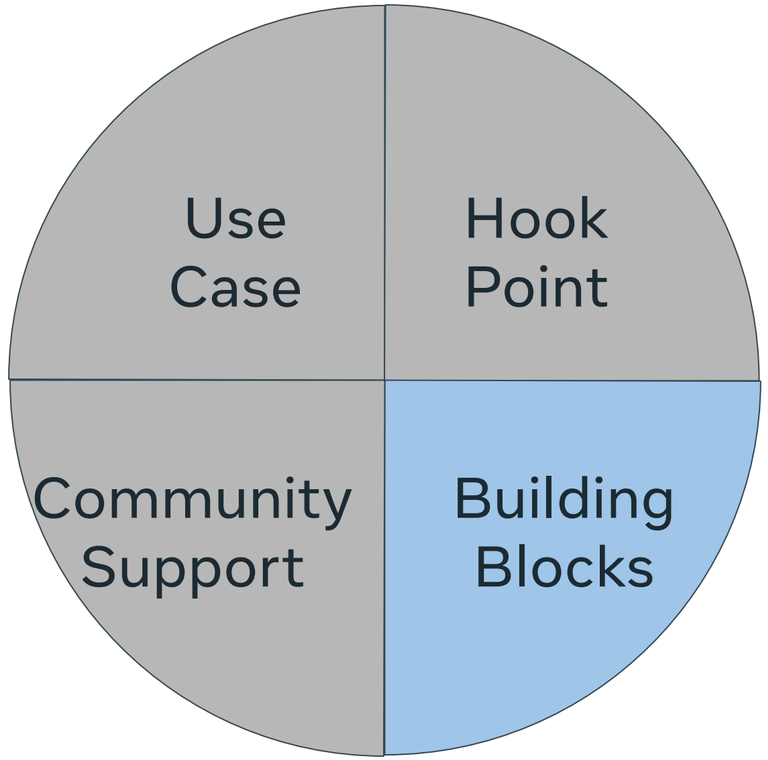
```
bpf_tramp_xxxx(struct file *file)
{
    ret = call(bpf_lsm_file_open+sizeof(nop));

    modified_ret = bpf_prog_xxxx(file, ret);

    return modified_ret;
}
```

BPF LSM building blocks

- BPF local storage
- New BPF kfuncs
  - Multiple kfuncs to read/write xattrs
  - `bpf_get_fsverity_digest`
  - `bpf_verify_pkcs7_signature`



Security data attached to kernel objects

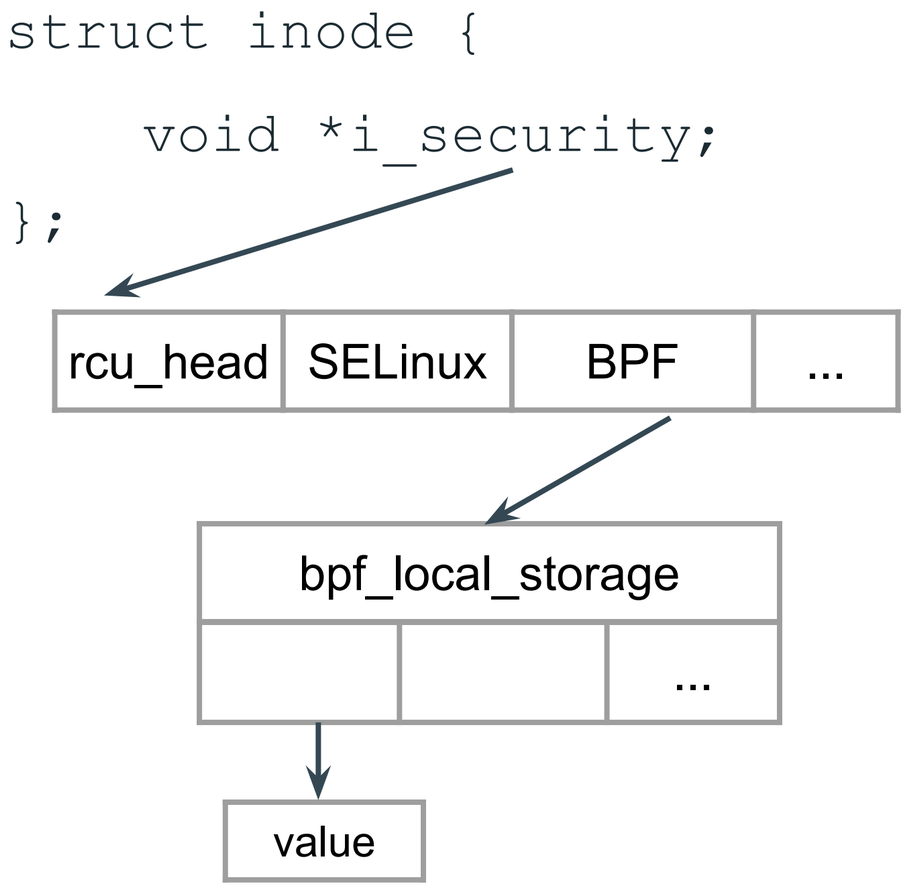
- Many LSMs use fix sized data for kernel object, e.g., `inode`, `task_struct`
- A data blob is allocated to serve all enabled LSMs
- The data blob has same life time as the kernel object it attaches to

```
struct inode {
    void *i_security;
};
```



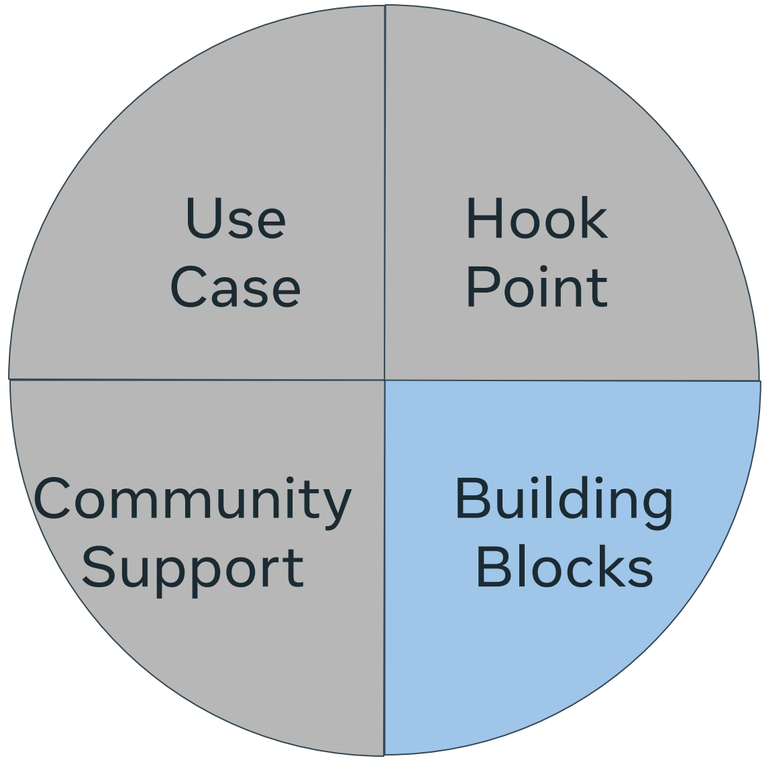
BPF local storage

- BPF LSM cannot use fixed sized data blob, because the size of per object data is not known ahead of time.
- BPF local storage was introduced to handle multiple maps
- Limitation: 3 pointer deref to access the value



BPF LSM building blocks

- BPF local storage
- New BPF kfuncs
  - Multiple kfuncs to read/write xattrs
  - `bpf_get_fsverity_digest`
  - `bpf_verify_pkcs7_signature`



Binary authorization with BPF LSM

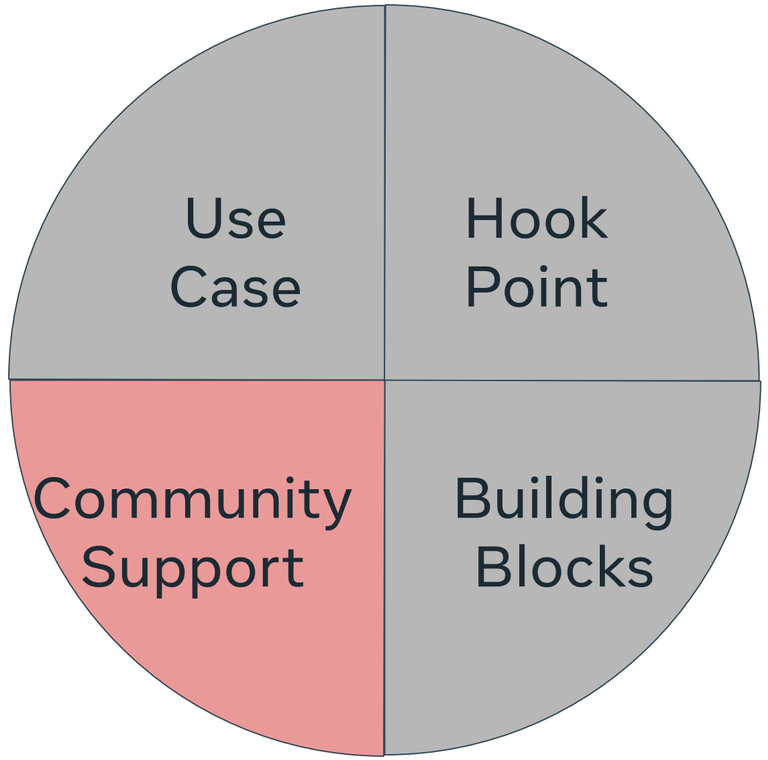
- Use case: Protect key resources against untrusted binary
  - e.g., only signed sshd binary can bind port 22
- Trusted build server signs the binary's fsverity digest (Merkle tree root hash) with private key
- The signature is stored in xattr
- User space security daemon loads the public key to keying

Binary authorization with BPF LSM, cont’d

```
SEC("lsm.s/bprm_committed_creds")
int BPF_PROG(bprm_committed_creds, const struct linux_binprm *bprm)
{   /* simplified to fit in one page */
    bpf_get_fsverity_digest(bprm->file, &digest);
    bpf_get_file_xattr(bprm->file, "user.sig", &sig);
    ret = bpf_verify_pkcs7_signature(&digest, &sig, keyring);
    /* If ret == 0, this binary has been signed by private key.
     * Set a flag in a BPF map.
     */
}
```

BPF LSM in the community

- `restrict_filesystems` in systemd
- Used by big companies
- Open source solution: Tetragon, more coming
- Improvements being shipped
- Use cases evolving quickly





01 What is BPF?

02 BPFize your kernel subsystem

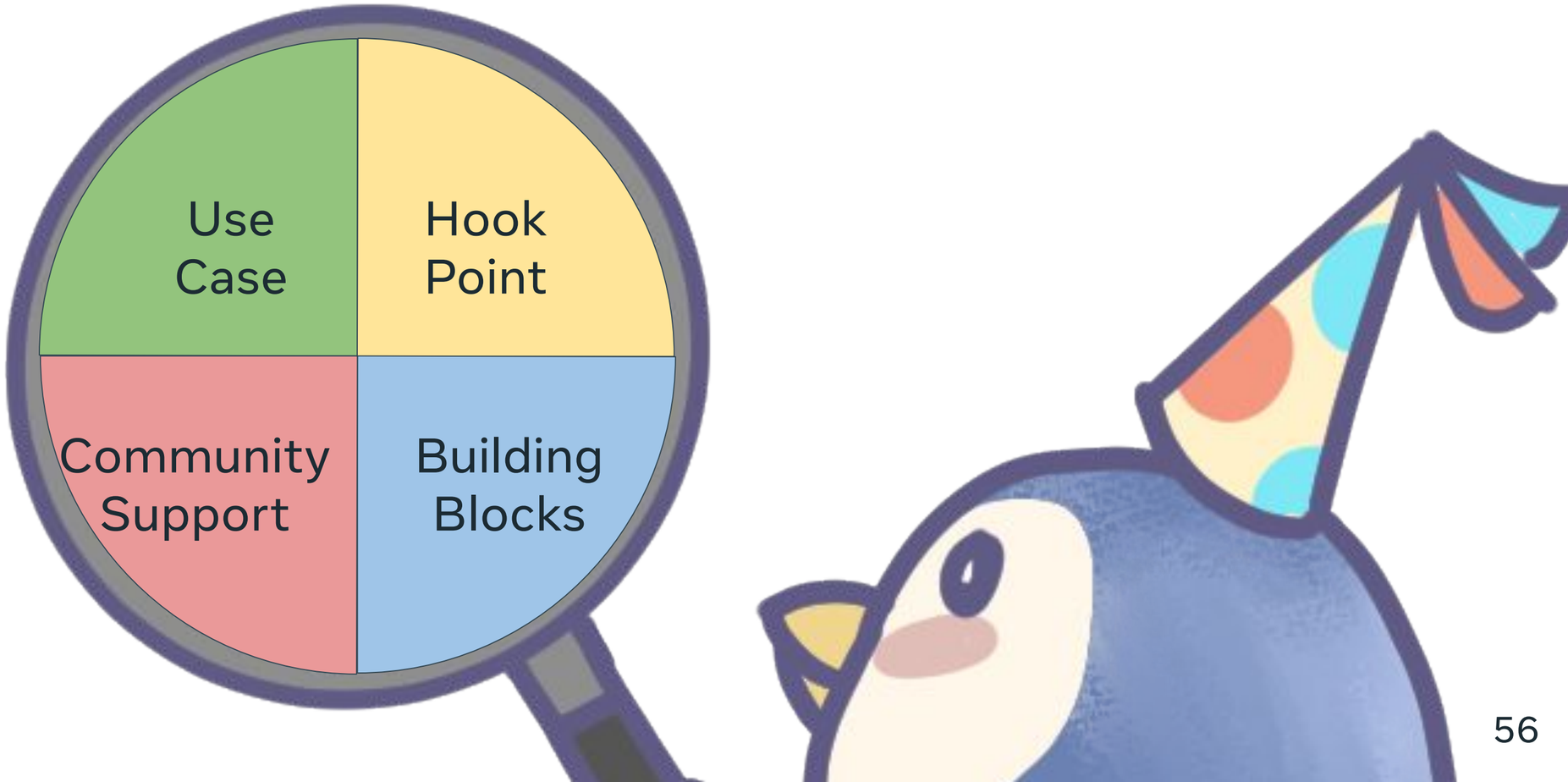
03 BPF LSM

04 fanotify-bpf

Limitation of LSM: local rules vs. global hooks

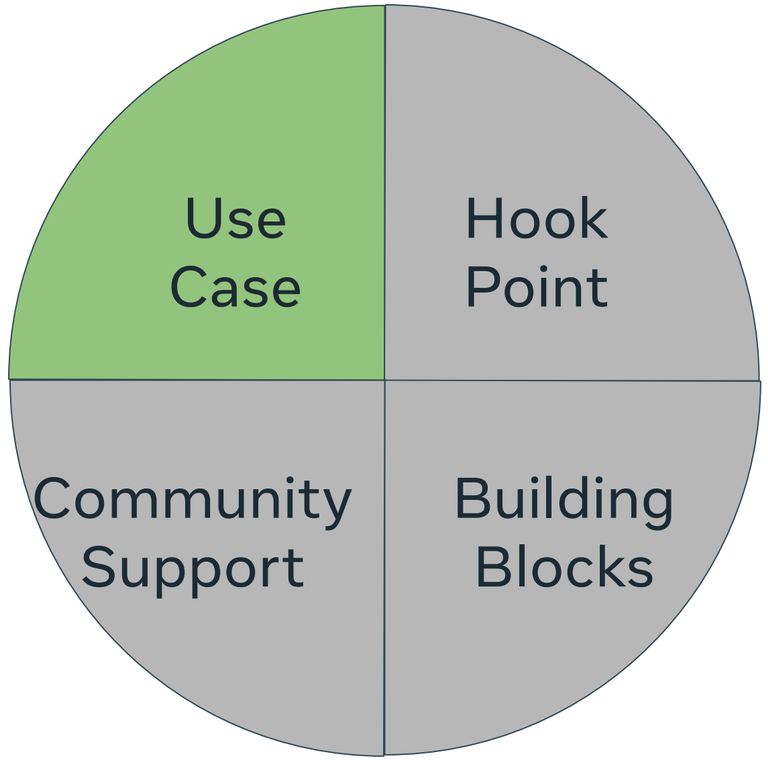
- Different access rules for different files
  - e.g., LSM only protects some key files
- LSM hooks are global
  - e.g., file_open hook is triggered on every file open

To Be Determined



Use case

- Apply security rules for a set of files.
- Specifically, for a subtree in VFS tree
- Alternative: BPF LSM



fanotify

- Filesystem notification system
- Also provides access control via permission events, used by Antivirus scanners
- fanotify can specify rules locally to a file/directory/superblock

fanotify permissions events

```
/* user process */
fd = open(file, ...);
```

```
/* monitor daemon */

read(fan_fd, buf, size);
metadata = (struct fanotify_event_metadata *)buf;
if (metadata->masks & FAN_OPEN_PERM){
    /* scan the file here, with metadata->fd */
    response.fd = metadata->fd;
    response.response = FAN_DENY;
    write(fan_fd, &response, sizeof(response));
}
```

```
/* fd == -EPERM */
```

fanotify: per inode/superblock rules

```
struct inode {
    __u32  i_fsnotify_mask; /* all events this inode cares about */
    struct fsnotify_mark_connector __rcu    *i_fsnotify_marks;
};

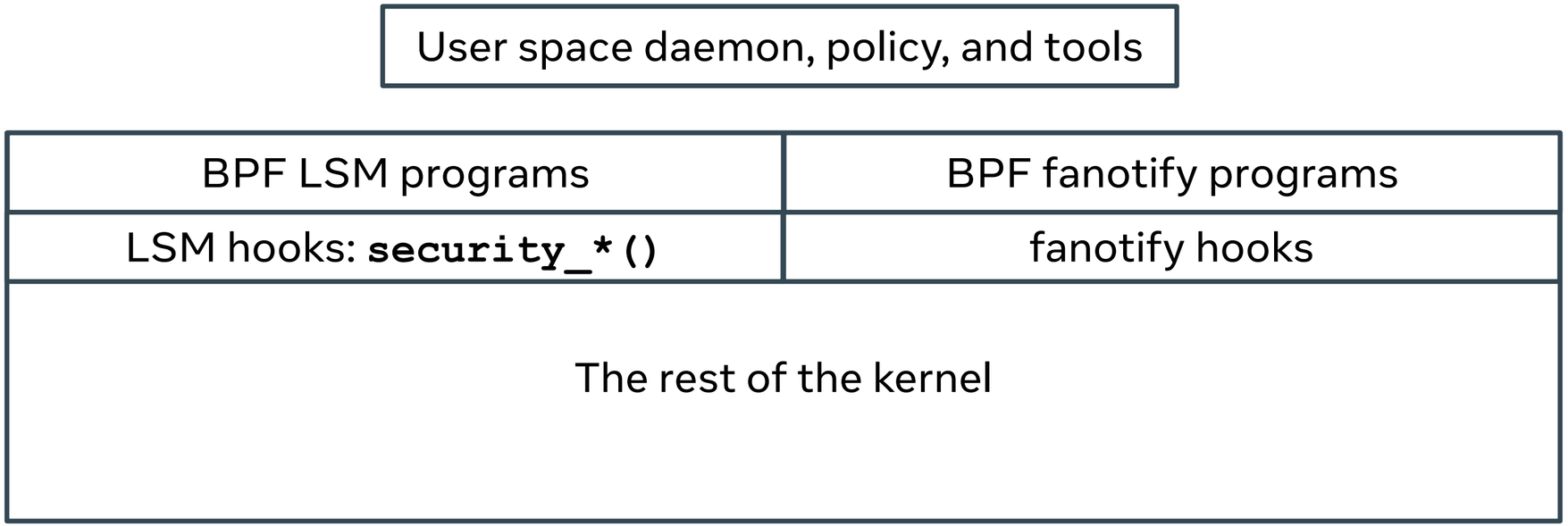
struct super_block {
    u32                     s_fsnotify_mask;
    struct fsnotify_sb_info *s_fsnotify_info;
};
```

fanotify: fast embedded filter

```
struct inode {
    __u32  i_fsnotify_mask; /* all events this inode cares about */
    struct fsnotify_mark_connector __rcu    *i_fsnotify_marks;
};

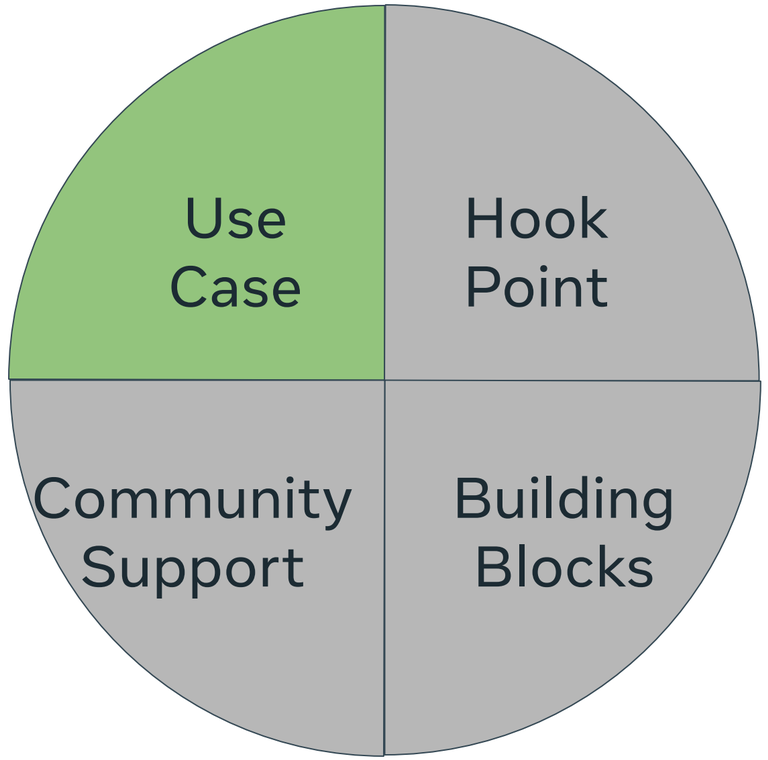
struct super_block {
    u32                     s_fsnotify_mask;
    struct fsnotify_sb_info *s_fsnotify_info;
};
```

BPF LSM + fanotify



fanotify-bpf use cases

- Apply security rules for a set of files.
- Specifically, for a subtree in VFS tree
- Alternative: BPF LSM
- Can also help Antivirus scanner

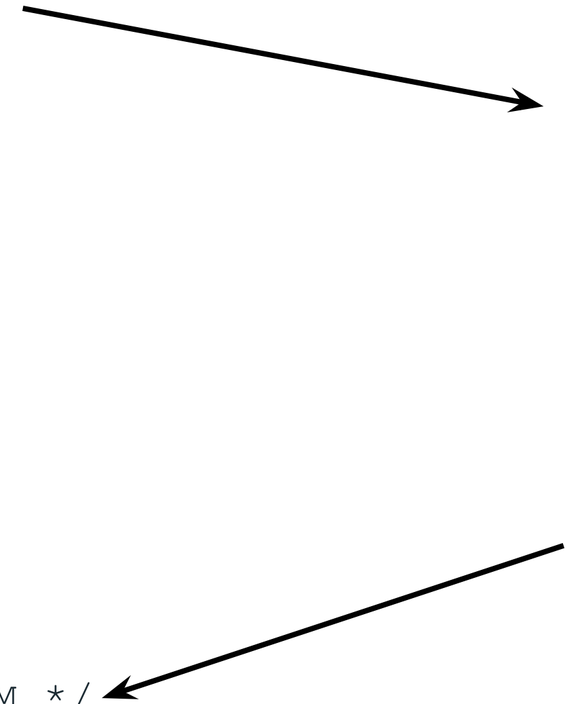


fanotify permission events

```
/* user process */
fd = open(file, ...);
```

```
/* monitor daemon */

read(fan_fd, buf, size);
metadata = (struct fanotify_event_metadata *)buf;
if (metadata->masks & FAN_OPEN_PERM){
    /* scan the file here, with metadata->fd */
    response.fd = metadata->fd;
    response.response = FAN_DENY;
    write(fan_fd, &response, sizeof(response));
}
```



```
/* fd == -EPERM */
```

fanotify with in-kernel fast path

```
/* user process */
fd = open(file, ...);
```

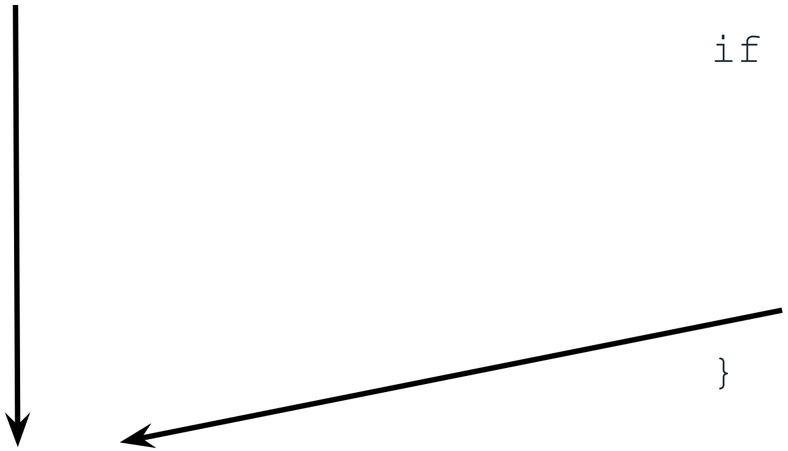


```
    fp_handler
```



```
/* monitor daemon */

read(fan_fd, buf, size);
metadata = (struct fanotify_event_metadata *)buf;
if (metadata->masks & FAN_OPEN_PERM){
    /* scan the file here, with metadata->fd */
    response.fd = metadata->fd;
    response.response = FAN_DENY;
    write(fan_fd, &response, sizeof(response));
}
```



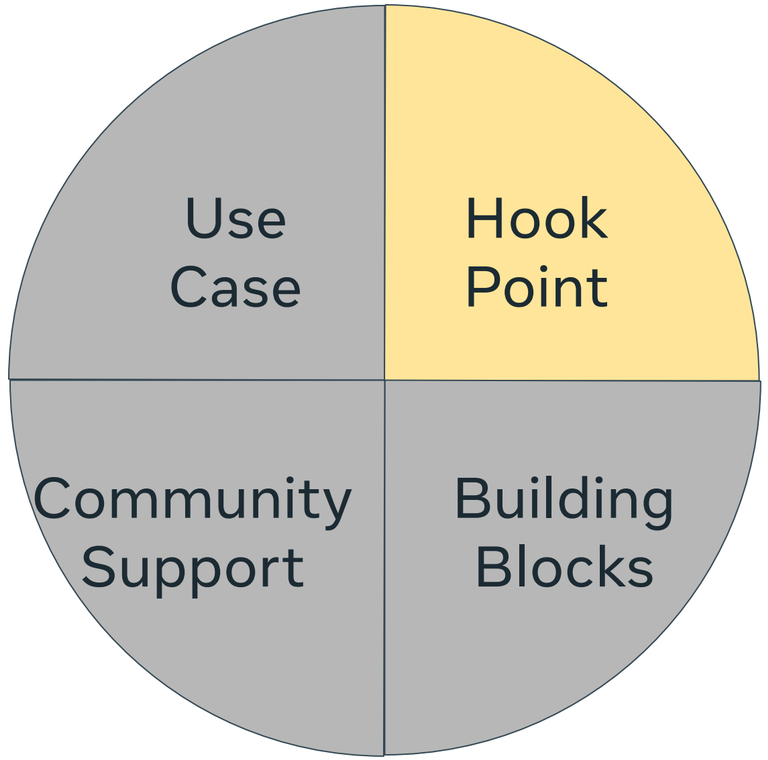
```
/* fd == valid or -EPERM  */
```

fanotify fastpath handler

```
struct fanotify_fastpath_ops {
    int (*fp_handler)(...);
    int (*fp_init)(...);
    void (*fp_free)(...);
    char name[FAN_FP_NAME_MAX];
    /* ... */
}
```

fanotify fastpath handler

```
struct fanotify_fastpath_ops {
    int (*fp_handler)(...);
    int (*fp_init)(...);
    void (*fp_free)(...);
    char name[FAN_FP_NAME_MAX];
    /* ... */
}
```

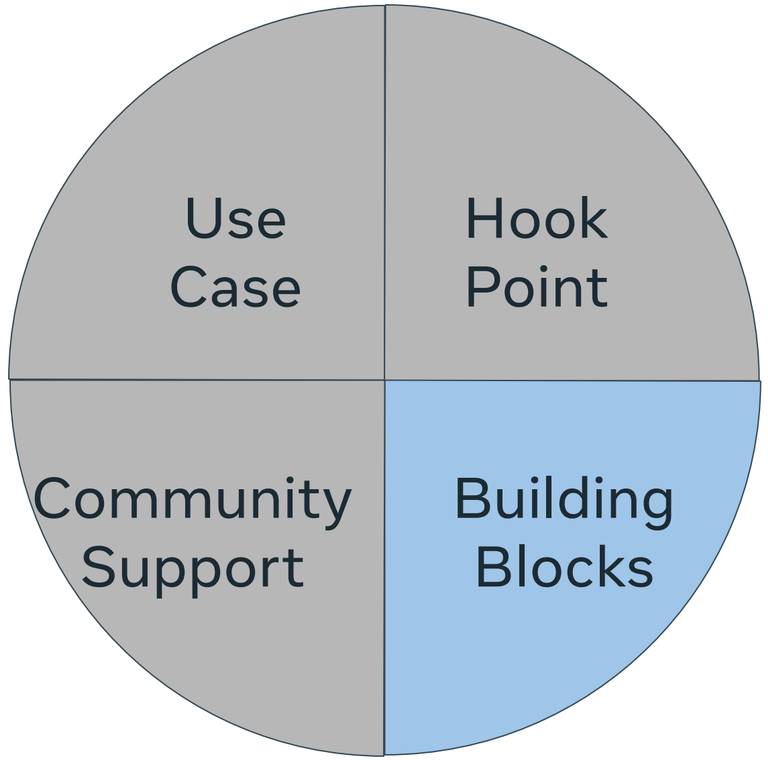


struct_ops

- Define an interface
  - tcp-bpf: congestion control
  - sched_ext: scheduler
- Multiple implementations of the interface
- Can use both kernel module or BPF
  - Kernel module: can use all exported symbols
  - BPF: safe

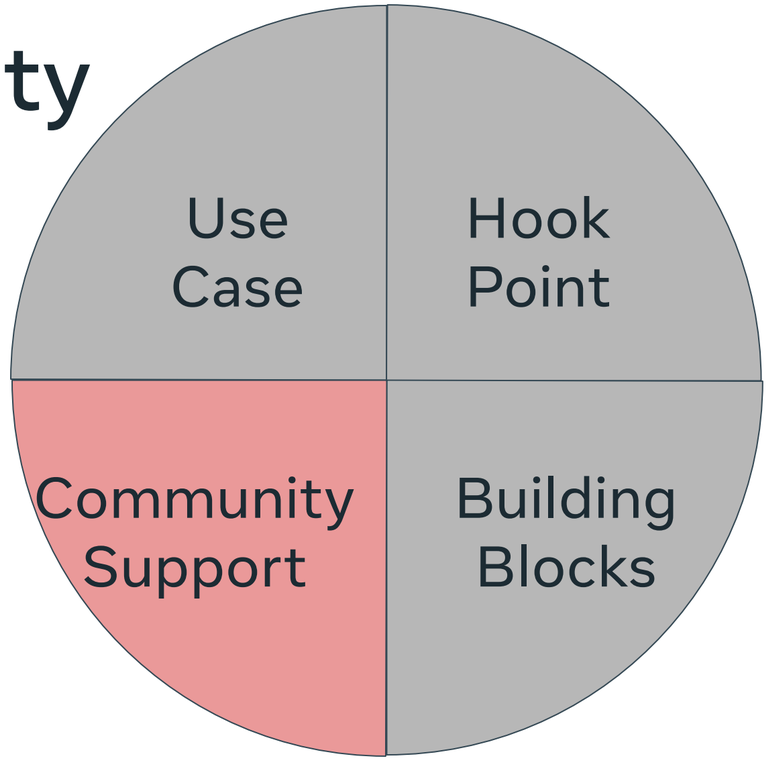
fanotify-bpf building blocks

- New kfunc `bpf_is_subdir(dentry, subroot)` to enable subtree rules
- Hold reference to `subroot` in BPF map

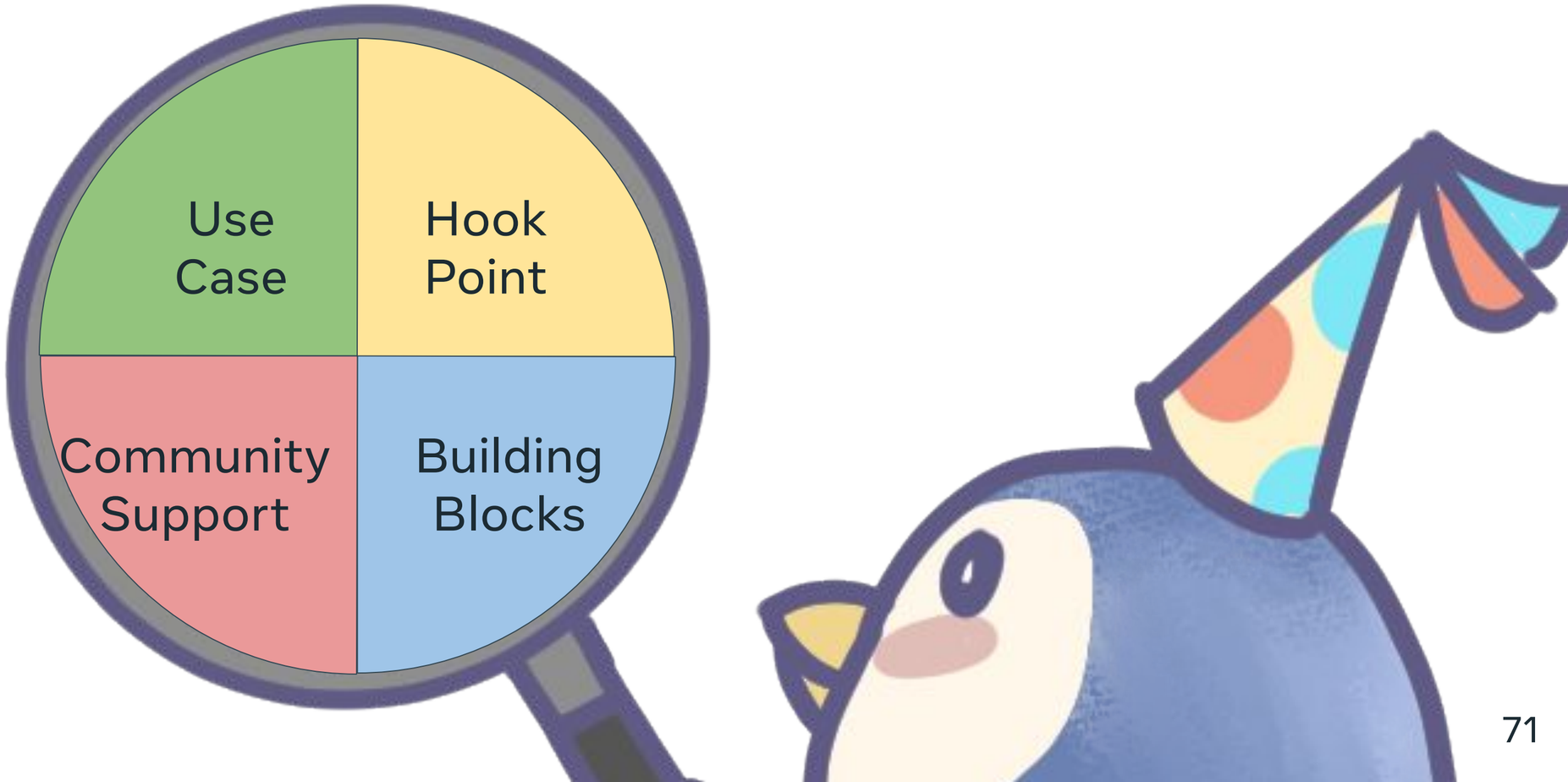


fanotify-bpf: engage the community

- Multiple rounds of patch reviews with VFS and BPF maintainers



fanotify-bpf



fanotify-bpf: not landed yet

- Issues with subtree monitoring
  - Requires per superblock rules, not really local
  - `bpf_is_subdir()` is not much faster than path walking in BPF
  - Holding `subroot` in BPF map could be risky, e.g., it may block umount
- Antivirus scanner may not need an in-kernel fastpath
- The alternative, BPF LSM, is good enough in most use cases



fanotify: fast embedded filter

```
struct inode {
    __u32  i_fsnotify_mask; /* all events this inode cares about */
    struct fsnotify_mark_connector __rcu    *i_fsnotify_marks;
};

struct super_block {
    u32                     s_fsnotify_mask;
    struct fsnotify_sb_info *s_fsnotify_info;
};
```

fanotify-bpf: not landed yet

- Issues with subtree monitoring
  - Requires per superblock rules, not really local
  - `bpf_is_subdir()` is not much faster than path walking in BPF
  - Holding `subroot` in BPF map could be risky, e.g., it may block umount
- Antivirus scanner may not need an in-kernel fastpath
- The alternative, BPF LSM, is good enough in most use cases



What’s next?

- Present fanotify-bpf at Kernel Recipes [DONE]
- Benchmark of BPF LSM vs. fanotify-bpf
- Better solution for subtree monitoring
- Let the use cases evolve




Merci!


Meta